Supplement to "When Does Admission Control Pay? The Memory Tier Sets the Cost of Concurrency in Agentic LLM Serving"

Anonymous Submission

Scope. This supplement carries the readings, the audit forensics, the version history and the retraction record behind the paper. The paper is self-contained: nothing here is needed to follow its argument, no claim rests on material that appears only here, and nothing here extends the paper's claims.

1 The full run table

Table I of the paper lists the rows the argument turns on; the table below is the complete run table, including the eight static caps the paper omits for space. Columns are the run, its wall clock, the mean running-request count during busy time, occupancy, deadline satisfaction rate, and the signed distance Δ from the static envelope at that wall clock. $\Delta > +1.0$ pp is above the line, $\Delta < -1.0$ pp below it, and the band between is a tie.

Policy	Wall (min)	Conc.	SLO (%)	Δ (pp)
Static caps				
no control	288.6	94.9	3.0	−87.62
cap1[‡]	527.4	1.00	94.8	*anchor*
cap2	304.2	1.99	89.5	−2.23
cap3 (fixed)	258.7	2.89	74.4	−11.77
cap3 (paired, dev)	246.4	2.97	79.3	−5.02
cap4	249.9	3.94	55.7	−29.17
cap5	260.9	4.89	27.1	−59.34
Static caps + HiCache				
cap2+HC	295.0	1.99	91.6	*anchor*
cap3+HC	236.4	2.93	81.1	−1.72
cap3 (paired, +HC)	233.8	2.97	81.5	−0.93
cap3+HC (repeat)	230.3	2.97	81.9	*anchor*
cap4+HC	213.0	3.93	63.4	−7.55
cap4+HC (repeat)	209.1	3.87	68.5	*anchor*
cap5+HC	227.6	4.74	47.3	−32.84
cap6+HC	207.8	5.85	28.9	*anchor*
cap8+HC	241.8	7.63	9.5	−74.10
Foyer controller variants				
scans the queue (2nd run)	262.6	2.41	88.9	+2.16
scans the queue	266.3	2.34	88.6	+1.30
scans the queue (3rd run)	271.7	2.29	88.4	+0.29
horizon=2 (2nd run)	274.2	2.22	88.7	+0.22
single-flight off (2nd run)	263.5	2.44	87.0	+0.13
horizon=2	271.0	2.28	87.7	−0.30
single-flight off	263.3	2.43	86.3	−0.55
horizon=1	255.8	2.58	84.8	−0.93
horizon=3 (base)	291.3	2.06	89.8	−1.24
resident base	265.9	2.41	85.9	−1.33
horizon=3 (2nd run)	293.2	2.07	89.6	−1.73
horizon=1 (3rd run)	274.4	2.37	85.3	−3.22
horizon=1 (repeat)	260.8	2.69	80.9	−5.58
horizon=0	264.2	2.64	79.8	−7.19
horizon=1, no hw	257.0	2.92	77.6	−8.31
no gate, no resv.	272.3	3.25	66.6	−21.62
running gate=4	282.3	3.03	64.3	−25.39
all throttles off	415.0	2.58	46.1	−47.10
Independent adaptive controller (§V)				
AIMD, paper params	326.8	2.17	79.4	−12.65
AIMD, retuned	264.6	3.87	52.8	−34.28

2 Per-arm attribution

Each controller arm is compared against the static run whose wall clock is closest to it, not against the static *front*: the classification describes an arm relative to a wall-clock peer. The six unused-memory rows also run fewer requests than their comparison, which describes what they did rather than demonstrating that another choice would have won.

arm	wall (min)	nearest static	occ. (%)	τ_1 (ms)	static occ. (%)	static τ_1 (ms)
foyer_sf0_r2	263.3	cap5	54.5	28.9	74.3	36.7
foyer_hz1	255.8	cap3_fixed	57.8	29.2	59.0	29.7
foyer_hc	291.3	pairB_cap2hc	49.2	28.7	41.7	27.0
foyer_hz2_r2	271.0	cap5	52.4	28.4	74.3	36.7
foyer_hz0_r2	264.2	cap5	58.1	30.2	74.3	36.7
foyer_bres_r2	265.9	cap5	56.2	29.5	74.3	36.7
foyer_breshz1_r2	257.0	cap3_fixed	62.7	30.5	59.0	29.7
foyer_nogate	272.3	cap5	62.1	31.5	74.3	36.7
foyer_rg4	282.3	pairB_cap2hc	62.3	32.2	41.7	27.0
foyfix_hc	415.0	cap2	39.2	34.8	41.6	27.4
aimd2	264.6	cap5	66.6	34.3	74.3	36.7
aimd2_paper_r3	326.8	cap2	44.3	31.8	41.6	27.4
foyer_scan	266.3	cap5	54.3	28.6	74.3	36.7

3 The controller's occupancy projection

The paper reports the ratio of the controller's projection to the occupancy the engine reports, across the twenty-seven controller runs that finished on this trace. The table below gives, per run, the median projection, the median occupancy the engine reports, and their ratio across that run's cycles. It lists every run in the file, which spans more than one trace; rows marked $*$ are the ones on the paper's trace.

run	cycles	budget	projection p50	occupancy p50	ratio
realhr_foyer_r2	2336	96360	80987	19624	4.13
realhr_foyer	2348	96360	80987	19695	4.11
pois200_v4_r2	1104	96359	52348	19082	2.74
thr25_push_r2	1008	96360	84387	40729	2.07
thr25_push	980	96360	80863	40076	2.02
pois200_foyer_push_25	931	96360	84183	42051	2.00
pois200_foyer_bres_r2	8029	96360	111730	56916	1.96
pois200_foyer_bres	8652	96358	109753	57665	1.90
pois200_foyer_sf0_r3	7952	96360	102492	54095	1.89
pois200_var_foyer	9844	96360	87127	46035	1.89
thr25_hwsf	1012	76074	72917	38690	1.88
pois200_foyer_scan_r2	7932	96360	105109	56199	1.87
p200_fix_r2	9193	96360	88993	47759	1.86
pois200_foyer	8834	96360	89390	47982	1.86
pois200_foyer_scan_r3	8155	96360	99324	53341	1.86
pois200_foyer_scan	8030	96360	101631	55151	1.84
pois200_foyer_sf0_r2	7945	96360	99953	55101	1.81
pois200_pairA_foyerhc	8436	96360	90121	50201	1.80
pois200_foyer_t85	9928	86217	81960	45700	1.79
pois200_foyer_hc_r2	8830	96360	87350	48848	1.79
pois200_foyer_hc	8784	96360	89073	49875	1.79
pois200_foyer_t90	9411	91289	85904	48310	1.78
pois200_foyer_hc_slru	8664	96360	88125	49778	1.77
pois200_pairB_foyerhc	8723	96360	88816	50806	1.75
thr25_nohw	1241	76074	63847	37318	1.71
pois200_foyer_hz2_r2	8150	96360	89125	53339	1.67
pois200_foyer_breshz1_r2	7756	96360	103455	63138	1.64
pois200_foyer_hz2_r3	8263	96360	89125	54652	1.63
load25c_budget3_oracle	2743	76074	55626	34228	1.63
pois200_foyer_hz1_r2	7837	96360	93828	58856	1.59
pois200_foyer_hz2	8968	96360	87905	55620	1.58
pois200_foyer_hz1	7722	96360	90935	58667	1.55
pois200_foyer_breshz1	9102	96360	94926	62373	1.52
pois200_foyer_hz1_r3	8279	96360	87048	57231	1.52
pois200_foyer_hz0_r2	7955	96360	82978	59001	1.41
pois200_foyer_nogate	8188	96360	88357	63160	1.40
pois200_foyer_rg4	8485	96360	85031	62854	1.35
pois200_foyer_rg2	6204	96358	76764	57263	1.34
pois200_foyer_hz0	8488	96360	85930	65271	1.32
load25c_budget3_agg_hc	1247	86217	72122	54945	1.31
budget200c_budget3_agg_hc	13529	86217	71823	57645	1.25
budget200c_budget3_r3	16388	76074	59593	47898	1.24
load25c_budget3_glob	1748	76074	62298	50089	1.24
pois200_foyer_rg3	1097	96360	83262	67041	1.24
load25c_budget3	1628	76074	62257	50175	1.24
load25c_budget3_hc	1753	76074	60005	48667	1.23
thr25_t95	1288	96360	72677	59399	1.22
thr25_base	1843	76074	54818	45036	1.22
thr25_nosf	1580	76074	61804	51061	1.21
budget200c_budget3_r2	15941	76074	60396	49982	1.21
load25c_budget3_t80	1630	81146	59500	49444	1.20
budget200c_budget3_timedout_591steps	8872	76074	60019	50723	1.18
load25c_budget3_sg0	2278	76074	49005	43310	1.13
load25c_budget3_zero	1381	76074	62113	59163	1.05
budget200c_budget3_zero	12511	76074	67233	64584	1.04
budget200c_budget3	35381	76074	11145	11086	1.01
pois200_foyer_r2	1657	96361	0	0	nan

4 The second accelerator

4.1 The three baseline attempts

run	attempt	pool tokens	wall min	slo pct	steps	failed steps	tau2 ms	occ p50 pct	occ mean pct	watch n	watch foreign	dirty span min	dirty before end min
a100_default	1	983728	88.5907	99.6997	999	0	18.6199	9.5183	13.2623	341	82	23.7215	23.8470
a100_default_r2	2	817527	148.0592	32.9329	999	2	222.6867	95.3428	81.2783	566	566	147.5941	147.8312

There were three attempts in the sense that a server was started three times. Only two produced readings, and they are the two rows above; the third was abandoned before the trace started, so it contributes no numbers. The second attempt is a reading of a contended host, not a baseline, and nothing in the paper uses it; it is listed because withholding it would make the first attempt look like the only one we made.

4.2 Static sweep and the controller on the second accelerator

run	machine	steps	wall min	pool tokens	Rbar busy	occ p50 pct	occ mean pct	tau1 ms
cap1	3090-24G	999	527.4	101432	0.998027	22.4	25.2	28.8
cap2	3090-24G	999	304.2	101432	1.271351	41.6	42.7	27.4
cap3_fixed	3090-24G	999	258.7	101432	1.534018	59.0	55.8	29.7
cap4	3090-24G	999	249.9	101432	1.650488	66.4	62.7	32.4
cap5	3090-24G	999	260.9	101432	1.840948	74.3	68.1	36.7
a100_cap1	A100-80G	999	403.4	971456	0.999712	2.1	2.3	21.0
a100_cap2	A100-80G	999	204.2	983728	1.238758	3.3	3.9	17.9
a100_cap3	A100-80G	999	136.5	983728	1.599936	4.5	5.7	16.7
a100_cap4	A100-80G	999	110.7	983728	1.970153	6.5	7.0	16.8
a100_cap5	A100-80G	999	96.4	983728	2.335561	8.0	8.5	17.8
a100_cap16	A100-80G	999	88.3	983697	3.223857	8.8	11.6	22.4
a100_cap8	A100-80G	999	90.1	943541	3.407727	12.3	12.9	24.4

5 The ready clock, per arm

The paper reports the engine clock (first token, timed from the engine) as primary, and notes that no arm moves from below the line to above it under the ready clock. The per-arm shifts are below: the ready clock charges the door, that is, it starts counting when a request is offered rather than when it is admitted.

run	slo engine pct	slo ready pct	delta pp	wait p50 ms	wait p90 ms	wait max s	front engine pct	front ready pct
cap1	94.8	76.2	-18.6	2	11033889	29977		
cap2	89.5	72.8	-16.7	2	6278927	17736	91.7	74.6
cap3_fixed	74.4	61.8	-12.6	2	4774019	13337	86.1	70.4
cap3_pd	79.3	64.9	-14.4	2	4159567	14238	84.3	69.0
cap4	55.7	46.4	-9.2	2	3868231	12976	84.8	69.4
cap5	27.1	24.2	-2.9	2	5192450	14292	86.5	70.7
pairB_cap2hc	91.6	74.5	-17.1	2	6337645	16062	91.6	74.5
cap3_hc	81.1	66.1	-15.0	1	4109412	12320	82.8	67.9
cap3_ph	81.5	66.3	-15.2	2	4041087	12835	82.4	67.6
cap4_hc	63.4	52.5	-10.9	2	1979637	10883	70.9	58.6
cap6_hc	28.9	25.2	-3.7	1	3290339	10420	28.9	25.2
cap3_hc_r2	81.9	67.2	-14.7	1	3779063	11663	81.9	67.2
cap4_hc_r2	68.5	56.7	-11.8	2	2058079	10807	68.5	56.7
cap5_hc	47.3	39.6	-7.7	2	2944805	12007	80.2	65.9
cap8_hc	9.5	9.1	-0.4	2	3970754	12085	83.6	68.5
foyer_hc_r2	89.6	72.0	-17.6	2	4789928	12533	91.3	74.3
foyer_hz1_r2	80.9	63.7	-17.2	2	4071608	10554	86.5	70.6
default	3.0	3.0	0.0	2	2	0	90.6	73.8
foyer_hc	89.8	72.4	-17.4	2	4982549	12437	91.0	74.1
foyer_hz1	84.8	68.2	-16.6	2	3792608	10290	85.7	70.1
foyer_hz1_r3	85.3	67.5	-17.8	2	4419753	11426	88.5	72.2
foyer_hz2_r2	87.7	69.9	-17.8	2	4424455	11217	88.0	71.8
foyer_hz2_r3	88.7	70.9	-17.8	2	4603060	11394	88.5	72.2
foyer_hz0_r2	79.8	61.8	-18.0	1	4540728	10766	87.0	71.0
foyer_bres_r2	85.9	68.1	-17.8	2	4396726	10958	87.2	71.2
foyer_breshz1_r2	77.6	58.1	-19.5	2	3361916	10238	85.9	70.2
foyer_nogate	66.6	48.2	-18.3	2	4127536	10693	88.2	71.9
foyer_rg4	64.3	45.0	-19.3	2	4567928	11263	89.7	73.1
foyer_scan	88.6	70.7	-17.9	2	4303268	10967	87.3	71.3
foyer_sf0_r2	86.3	69.5	-16.8	2	4098985	10847	86.8	70.9
foyer_scan_r2	88.9	71.8	-17.1	2	4394751	10675	86.7	70.8
foyer_scan_r3	88.4	70.8	-17.6	1	4470955	11291	88.1	71.9
foyer_hc_slru	89.5	72.1	-17.4	2	4805924	12303		
foyer_sf0_r3	87.0	70.4	-16.6	2	3950338	10847	86.9	70.9
foyfix_hc	46.1	36.5	-9.6	2	7883727	19381	93.2	75.4
aimd2	52.8	39.5	-13.2	2	4386996	10410	87.0	71.1
aimd2_paper_r3	79.4	61.6	-17.8	2	6360456	14723	92.0	74.7

6 Per-arm re-derivation

The paper's claim is not that admission control is useless but that the one arm above the static envelope is a change to the *release* rule, while every arm that changes the *admission* decision is not. The per-arm distances behind that sentence are below; Δ is the signed distance from the static envelope at the arm's own wall clock, so $\Delta > +1.0$ pp is above the line and $-1.0 \le \Delta \le +1.0$ is the tie band. The three `scan` rows are the same release-rule change run three times: two are above the line and the third lands inside the band, which is why the paper reports it as two of three rather than as a crossing.

run	wall min	slo pct	delta pp	verdict
foyer_hz1	255.8371	84.7848	-0.9275	TIE
foyer_breshz1_r2	256.9884	77.5776	-8.3075	BELOW
foyer_hz1_r2	260.7962	80.8809	-5.5755	BELOW
foyer_scan_r2	262.6125	88.8889	2.1600	ABOVE
foyer_sf0_r2	263.3403	86.2863	-0.5519	TIE
foyer_sf0_r3	263.4591	86.9870	0.1310	TIE
foyer_hz0_r2	264.2349	79.7798	-7.1926	BELOW
aimd2	264.6104	52.7528	-34.2759	BELOW
foyer_bres_r2	265.8658	85.8859	-1.3312	BELOW
foyer_scan	266.3133	88.5886	1.3044	ABOVE
foyer_hz2_r2	270.9757	87.6877	-0.2961	TIE
foyer_scan_r3	271.7073	88.3884	0.2948	TIE
foyer_nogate	272.3026	66.5666	-21.6163	BELOW
foyer_hz2_r3	274.2295	88.6887	0.2167	TIE
foyer_hz1_r3	274.4405	85.2853	-3.2184	BELOW
foyer_rg4	282.3023	64.2929	-25.3904	BELOW
foyer_hc	291.2765	89.7898	-1.2400	BELOW
foyer_hc_r2	293.2012	89.5896	-1.7290	BELOW
aimd2_paper_r3	326.8323	79.3794	-12.6507	BELOW
aimd2_paper	364.2547	81.3814	-11.1644	BELOW

7 Co-tenancy on the measurement host

An audit flags every run that overlapped three or more others for at least an hour. Fourteen runs are flagged — twelve of the main trace and two of the second workload; the incidents that produced those windows also killed four runs outright, and those four appear in no table of ours.

The flag is a risk marker, not a damage measurement. Engine-side counters (running count, pool occupancy, evictions, recomputation) are indistinguishable between flagged and clean runs, so the audit can only be run on the client-side per-request decode interval τ_2. Where damage is legible it is large: the worst-affected arm reads 33.3 ms when flagged against 24.8 ms on a clean re-run, while 17 clean runs of the same controller family read 23.8–27.5 ms.

Three within-run comparisons, each entirely inside one run, are the strongest evidence we have that the effect is real: for `cap1`, the per-half-hour median τ_2 is 20.3–20.9 ms over the first five and a half hours and 21.3/27.1/21.1 ms over the three and a half hours after two neighbours start; for `t85` it is 29.8 ms in the half hour before a neighbour leaves and 22.3–22.6 ms over the four and a half hours after; for `rl99` it is 32.4 ms against 25.2/25.1 ms. Same run, same policy, same trace — what changes is who else is on the machine.

Volume columns do not see the damage: the one arm we have two readings for reuses 0.6943 of its prompt when flagged and 0.6904 alone, both inside the 0.4511–0.6981 range of the twenty-eight host-tier runs we have two-use data for, while its τ_2 moves from 31.3 to 24.8 ms. That is suggestive, not proof: one arm is one comparison.

8 Controller artifact version history

The runs record four distinct hashes of the controller module, and none of them matches the copy in the repository. No single revision therefore reproduces every number we report. The arms the audit flags are not a scattered subset: they are *all four* runs recorded under one of those copies, and a second small copy does not break the tie — so revision and co-tenancy cannot be separated by any comparison in our data. We report the confound rather than resolve it. The re-runs changed the contention windows and the controller revision together.

The copy most arms ran under is still on the measurement host, so its divergence from the repository can be re-derived rather than trusted: it differs in two places — it runs single-flight on the first queue entry rather than scanning the whole queue, and it writes the candidate's own context above the floor it logs as `base`. The narrow experiment now exists and the paper reports it: same copy, that one expression changed, and it is the arm that crosses the line. Its own comment records what the expression removed: when the head of the queue is too large to fit, it blocks every smaller session behind it while the engine idles.

One reservation applies to that comparison: the harness scripts that launch the runs are not identical — the baseline repeats and the scan arm record a different hash of it, as do the two `horizon`=2 runs and the two single-flight-off runs (the scan arm's own repeat records the same one). Earlier versions were overwritten in place, so we can neither diff them nor show the launcher changed nothing else. The recorded parameters and build do match the repository, and the controller source differs only in the expression and a comment block named above.

9 The seven retractions, with their evidence

Each entry names where it can be checked; several are checkable directly against a data file, and one against a generator's own run-time assertion. The audit files themselves are working records, not authorities.

1. **The denominator of the step cost.** The old rule divided τ by the whole-run mean running count while multiplying by busy time. The two errors cancelled, so the two measurement methods appeared to agree. The bias is exactly the run's idle fraction — 36.3% for `cap1`. `decomp.csv` carries the four columns that recompute it.

2. **A median-based reading of the tool wait.** The old conclusion rested on a median of 0.34 s and concluded that sessions never leave the engine. The demanded mean is 15.82 s, the maximum 300 s, and 18.0% of the 799 round transitions wait more than ten seconds; the controller's run measures 16.85 s. `toolwait.csv` carries all three denominators.

3. **An oracle-monotonicity claim.** It rested on an arm later shown to be implemented incorrectly, and that arm may not be cited. The rewrite was never re-run, so the monotonicity is *unmeasured*.

4. **Double charging in the accounting.** The old rule added `pausing_tokens` on top of `measured`, charging twice for sessions already in the engine. It is falsified not by argument but by a log: `measured` 97,180 plus `pausing` 15,700 is 112,880, above the pool of 101,432, and the engine cannot hold more unreclaimable KV than the pool.

5. **A monotone idle-rate claim.** The old text said the idle share falls monotonically with concurrency. On the busy-conditional convention, the arm with no gate and no reservation idles 20.7% at $\overline{R}_{\text{busy}} = 1.68$ while `cap4` idles 10.4% at 1.65 — more concurrency, less idle.

6. **A refutation that was itself refuted.** An early revision dismissed the eviction-then-recompute mechanism with two checks that were each wrong: the ratio of total prefill tokens to planned prefill is 1.00 by construction, because `prompt_tokens_total` includes the cached part, so it cannot detect recomputation; and "no withdrawals" reported by the engine counts mid-flight preemption of running requests, not radix eviction.

7. **An inverted audit predicate.** The stall audit required `valve` to be true, while in the logs `valve` true means the valve *is* holding admissions — so the windows it reported were the valve working as designed. Measured inside one run, 0.5% of cycles admit when the valve is true against 9.8% when it is false. Corrected, one arm has one window, 166.6 s, 0.93% of the sampling span. The direction is asserted at run time by `gen_stall.py`.

10 Per-run idle decomposition

The paper reports that the remedy for idle time works and does not pay: it is not what improves the criterion. The per-run decomposition behind that is below. The identity is span = idle + busy, not wall = idle + busy: the sampler starts before the requests and stops after them, so the wall clock is short by an offset that differs per arm.

The structural remedy — giving the engine work to do during a wait — is what the single-flight-off arm tests; it reduces idle and does not move the criterion.

run	wall min	span min	idle min	busy min	idle pct	running	running all	step back ms	step perreq ms	excess pct
cap6_hc	207.8	213.8	13.4	200.4	6.3	1.87	1.75	31.8	27.6	15.0
cap4_hc_r2	209.1	215.8	15.8	200.0	7.3	1.76	1.63	29.9	25.6	16.8
cap4_hc	213.0	219.0	16.1	202.8	7.4	1.73	1.6	29.8	25.7	16.0
cap5_hc	227.6	233.7	25.9	207.8	11.1	1.75	1.55	30.8	26.8	15.1
cap3_hc_r2	230.3	236.4	20.1	216.3	8.5	1.53	1.4	28.1	24.8	12.9
cap3_ph	233.8	239.8	27.1	212.7	11.3	1.56	1.38	28.1	24.8	13.5
cap3_hc	236.4	242.5	24.1	218.4	9.9	1.51	1.36	28.0	24.5	14.3
cap8_hc	241.8	247.9	29.4	218.5	11.9	1.85	1.63	34.4	29.0	18.5
cap3_pd	246.4	252.5	26.4	226.2	10.4	1.51	1.35	29.0	24.6	17.9
cap4	249.9	257.6	26.7	230.9	10.4	1.65	1.48	32.4	25.8	25.8
foyer_hz1	255.8	261.9	36.7	225.2	14.0	1.52	1.31	29.2	24.6	18.5
foyer_breshz1_r2	257.0	263.0	46.4	216.6	17.7	1.65	1.36	30.5	25.5	19.6
cap3_fixed	258.7	264.7	37.1	227.6	14.0	1.53	1.32	29.7	24.9	19.1
foyer_hz1_r2	260.8	266.9	33.6	233.4	12.6	2.1	1.83	41.6	25.0	66.3
cap5	260.9	266.9	32.4	234.5	12.1	1.84	1.62	36.7	31.2	17.7
foyer_scan_r2	262.6	268.8	37.1	231.6	13.8	1.45	1.25	28.6	24.4	17.3
foyer_sf0_r2	263.3	269.4	35.4	234.1	13.1	1.45	1.26	28.9	24.3	19.1
foyer_sf0_r3	263.5	269.5	39.4	230.1	14.6	1.45	1.24	28.5	24.3	17.0
foyer_hz0_r2	264.2	270.5	45.6	224.8	16.9	1.58	1.31	30.2	25.5	18.5
aimd2	264.6	271.1	34.3	236.8	12.7	1.71	1.49	34.3	25.5	34.7
foyer_bres_r2	265.9	272.5	37.9	234.6	13.9	1.48	1.28	29.5	24.8	19.0
foyer_scan	266.3	272.3	38.7	233.6	14.2	1.44	1.23	28.6	24.4	17.1
foyer_hz2_r2	271.0	277.0	36.8	240.3	13.3	1.39	1.21	28.4	24.0	18.4
foyer_nogate	272.3	278.4	57.5	220.9	20.7	1.68	1.33	31.5	25.6	22.9
foyer_hz2_r3	274.2	280.2	38.8	241.3	13.9	1.4	1.21	28.8	24.0	19.8
foyer_hz1_r3	274.4	280.6	49.9	230.6	17.8	1.51	1.24	29.5	25.0	18.2
foyer_rg4	282.3	288.4	65.6	222.8	22.7	1.69	1.3	32.2	27.0	19.6
default	288.6	294.8	33.6	261.2	11.4	1.91	1.69	42.5	39.2	8.4
foyer_hc	291.3	297.5	47.4	250.1	15.9	1.35	1.14	28.7	23.8	20.7
foyer_hc_r2	293.2	299.2	49.3	249.9	16.5	1.36	1.14	29.0	24.1	20.5
pairB_cap2hc	295.0	303.3	50.1	253.2	16.5	1.25	1.05	27.0	22.7	19.0
cap2	304.2	310.4	57.0	253.5	18.4	1.27	1.04	27.4	22.8	19.9
pairA_cap2hc	304.5	310.6	55.3	255.3	17.8	1.26	1.03	27.3	22.9	19.2
aimd2_paper_r3	326.8	333.3	54.0	279.4	16.2	1.34	1.12	31.8	23.7	34.1
aimd2_paper	364.3	370.7	89.7	281.1	24.2	1.35	1.03	32.4	24.4	32.6
foyfix_hc	415.0	421.0	176.4	244.7	41.9	1.68	0.97	34.8	27.5	26.5
cap1	527.4	533.6	193.6	340.0	36.3	1.0	0.64	28.8	20.6	39.7
foyer_scan_r3	271.7	277.8	39.0	238.8	14.0	1.43	1.23	29.0	24.8	17.0
foyer_hc_slru	288.9	295.0	49.2	245.8	16.7	1.38	1.15	28.8	23.9	20.8

11 Scripts, data files, and the five independent checks

Every number in the paper has a row in one of the data files under `paper/data/`. Front and signed distances are recomputed from the raw per-step logs over a full-precision run table; the criterion is recomputed from its own elasticity generator. Each data file is produced under `scripts/lab/` from the raw per-run logs (`steps.jsonl`, `summary.json`, `metadata.json`, `metrics.csv`, `controller.jsonl`, `admissions.jsonl`). Two of the generators write two files each, and three files — the co-tenancy ledger and the two completion-counted tables — are carried rather than regenerated, because their content depends on which runs had finished when they were produced.

Five quantities the paper turns on are re-derived by five checks that share no code with the generators, each reproducing the table it checks:

- front and signed distance (`independent_frontier_check.py`);
- the segmented elasticity (`independent_elasticity_check.py`);
- the per-arm attribution (`independent_attribution_check.py`);
- the busy/idle decomposition (`independent_decomp_check.py`);
- realized reuse against the trace's bound (`independent_realization_check.py`).

`scripts/lab/regen_all.sh` regenerates every data file from the raw logs in a temporary copy of the tree and compares the result byte for byte against the delivered files; it does not write to the delivered tree. Three files are carried as frozen inputs rather than regenerated — the co-tenancy ledger and two tables whose content depends on which runs had finished when they were produced — and the script says so where it skips them.

The consistency checker runs several hundred assertions over the paper, the supplement and the data files. Its exit status is the deliverable: every number quoted in the text must be found, and a value that no longer matches its data file is a failure.

12 Pool shortfall and comparability

Every run is guarded: a pool that does not come out at $101{,}432 \pm 20$ voids the run rather than recording it. The pool can come out short because the engine rounds the KV pool down to a multiple of its page size and because a share of the pool is reserved before the trace starts; a short pool is not a smaller experiment but a different one, since occupancy is quoted as a share of the pool and the admission decisions compare a projection against a budget derived from it. On the second accelerator three runs came up off the nominal 983,728 — 971,456, 943,541 and 983,697 — and none of the three affects the column it is used for.